Computer Architecture HW 3 – Floating Point Arithmetic

1)a) -13.25_{10} (single precision) = 0xC1540000.

$|-13.25_{10}| = 13.25_{10} = 13 + 0.25 = 8 + 4 + 1 + ¼ = 1101.01_2 = 1.10101_2 * 2^3$.

Since, the number is negative, the sign bit = 1.

In the single precision format, the bias = 127 and therefore, the exponent = 3 + 127 = 130 = 128 + 2 = 10000010.

The mantissa is now 1010100…

Notice that we ignore the first digit before the decimal point (1) as that is always fixed in this format.

Therefore, we can write -13.25_{10} as 1 10000010 1010100…$_2$ = 1100 0001 0101 0100 … = 0xC1540000.

b)0.1_{10} (single precision) = 0x3DCCCCCD.

$0.1_{10} * 2 = 0.2_{10}$, $0.2_{10} * 2 = 0.4_{10}$, $0.4_{10} * 2 = 0.8_{10}$, $0.8_{10} * 2 = 1.6_{10}$, $0.6_{10} * 2 = 1.2_{10}$, $0.2_{10} * 2 = 0.4_{10}$

$0.1_{10} = 0.000110011\ldots_2 = 1.100110011..{}_2 * 2^{-4}$.

Since, the number is positive, the sign bit = 0.

In the single precision format, the bias = 127 and therefore, the exponent = – 4 + 127 = 123_{10} = 64 + 32 + 16 + 8 + 2 + 1 = 01111011_2.

The mantissa is now 10011 0011 0011 0011 0011 00.

Notice that we ignore the first digit before the decimal point (1) as that is always fixed in this format.

Therefore, we can write 0.1_{10} as 0 01111011 $10011001100110011001100_2$ = 0011 1101 1100 1100 1100 1100 1100 1100…$_2$ ~ 0x3DCCCCCD. (Notice that the pattern 1100 or C repeats infinitely and hence while converting it to the floating-point representation, the infinitely many C's are rounded up to a D after 32 bits or the 8th digit in the hexadecimal representation).

c)156.75_{10} (double precision) = 0x4063940000000000.

$156.75_{10} = 128 + 16 + 8 + 4 + 0.5 + 0.25 = 10011100.11_2 = 1.001110011_2 * 2^7$.

Since, the number is positive, the sign bit = 0.

In the double precision format, the bias = 1023 and therefore, the exponent = 7 + 1023 = 1030_{10} = 1024 + 4 + 2 = 10000000110_2.

The mantissa is now 001110011000…

Notice that we ignore the first digit before the decimal point (1) as that is always fixed in this format.

Therefore, we can write 156.75_{10} as 0 10000000110 001110011000…$_2$ = 0100 0000 0110 0011 1001 1000…$_2$ = 0x4063940000000000.

d)–0.0078125_{10} (double precision) = 0xBF80000000000000.

|–0.0078125_{10}| = 0.0078125_{10}.

0.0078125_{10}*2 = 0.015625_{10}, 0.015625_{10}*2 = 0.03125_{10}, 0.03125_{10}*2 = 0.0625_{10}, 0.0625_{10}*2 = 0.125_{10} = 2^{-3}.

Therefore, 0.0078125_{10} = $2^{-3}/2^4$ = 2^{-7} = 1.00000*2^{-7}.

Since, the number is positive, the sign bit = 1.

In the double precision format, the bias = 1023 and therefore, the exponent = –7 + 1023 = 1016_{10} = 512 + 256 + 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 – 7 = 512 + 256 + 128 + 64 + 32 + 16 + 8 = 01111111000_2.

The mantissa is now 000…

Notice that we ignore the first digit before the decimal point (1) as that is always fixed in this format.

Therefore, we can write –0.0078125_{10} as 1 01111111000 0000…$_2$ = 1011 1111 1000 0000..$_2$ = 0xBF80000000000000.

2)a)0xC1200000 (single precision) = –10.0_{10}.

0xC1200000 = 1100 0001 0010 0000….$_2$ = 1 10000010 0100…$_2$.

Sign bit = 1 which implies that the number is negative.

10000010_2 = 128_{10} + 2_{10} = 130_{10}.

For the single precision format, Bias = 127_{10}.

Actual exponent = Exponent – Bias = 130 – 127 = 3_{10}.

Mantissa = 0100…$_2$.

Therefore, the number is –1.01_2*2^3 = –1010_2 = –10_{10}.

b)0x3F800000 (single precision) = 1.0_{10}.

0x3F800000 = 0011 1111 1000 0000 ….$_2$ = 0 01111111 000 0000 ….$_2$.

Sign bit = 0 which implies that the number is positive.

01111111_2 = 64 + 32 + 16 + 8 + 4 + 2 + 1 = 127_{10}.

For the single precision format, Bias = 127_{10}.

Actual exponent = Exponent – Bias = 127 – 127 = 0_{10}.

Mantissa = 0000…$_2$.

Therefore, the number is $1.00_2*2^0 = 1_{10}$.

c)0xBFF0000000000000 (double precision) = -1.0_{10}.

0xBFF0000000000000 = 1011 1111 1111 0000 $\ldots_2$ = 1 01111111111 0000 $\ldots_2$.

Sign bit = 1 which implies that the number is negative.

$01111111111_2 = 512 + 256 + 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 1023_{10}$.

For the single precision format, Bias = 1023_{10}.

Actual exponent = Exponent – Bias = $1023 - 1023 = 0_{10}$.

Mantissa = 0000$\ldots_2$.

Therefore, the number is $-1.00_2*2^0 = -1_{10}$.

d)0x4024000000000000 (double precision) = 10.0_{10}.

0x4024000000000000 = 0100 0000 0010 0100 $\ldots_2$ = 0 10000000010 0100 $\ldots_2$.

Sign bit = 0 which implies that the number is positive.

$10000000010_2 = 1024 + 2 = 1026_{10}$.

For the single precision format, Bias = 1023_{10}.

Actual exponent = Exponent – Bias = $1026 - 1023 = 3_{10}$.

Mantissa = 0100$\ldots_2$.

Therefore, the number is $1.0100_2*2^3 = 1010_2 = 10_{10}$.

3)If A = 0x41480000 (single precision), B = 0xC0700000 (single precision),

Then, A + B = 0x4021800000000000 (double precision).

A = 0x41480000 = 0100 0001 0100 1000$\ldots_2$ = 0 10000010 1001000$\ldots_2$.

Sign bit = 0, and hence A is positive.

$10000010_2 = 128 + 2 = 130$, $130 - 127 = 3_{10}$. (Bias for single precision format is 127)

Therefore, A = $1.1001000_2*2^3 = 110.01_2*2^1$.

B = 0xC0700000 = 1100 0000 0111 0$\ldots_2$ = 1 10000000 1110$\ldots_2$.

Sign bit = 1, and hence B is negative.

$10000000_2 = 128$, $128 - 127 = 1_{10}$. (Bias for single precision format is 127)

Therefore, B = -1.1110_2*2^1.

Therefore, A + B = $110.01_2*2^1 - 1.1110_2*2^1 = (110.01_2 - 1.1110_2)*2^1 = 100.0110_2*2^1 = 1000.11_2 = 1.00011*2^3$.

Converting the sum to double precision format, we get

Sign bit = 0 as the number is positive.

Exponent = Actual exponent + Bias. Bias for the double precision format = 1023.

Exponent = 3 + 1023 = 1026 = 1024 + 2 = 10000000010_2.

Mantissa = $00011000\ldots_2$.

Therefore, the number in double precision format is 0 10000000010 $00011000\ldots_2$ = 0100 0000 0010 0001 $1000\ldots_2$ = 0x4021800000000000.

4)If A = 0x4039000000000000 (double precision), B = 0xC008000000000000 (double precision), then A * B = = 0xC2960000 (single precision).

A = 0x4039000000000000 = 0100 0000 0011 1001 $0000\ldots_2$ = 0 10000000011 $10010\ldots_2$.

Sign bit = 0, and hence A is positive.

10000000011_2 = 1024 + 2 + 1 = 1027, 1027 – 1023 = 4_{10}. (Bias for double precision format is 1023)

Therefore, A = 1.1001_2*2^4.

B = 0xC008000000000000 = 1100 0000 0000 $1000\ldots_2$ = 1 10000000000 $1000\ldots_2$.

Sign bit = 1, and hence B is negative.

10000000000_2 = 1024, 1024 – 1023 = 1_{10}. (Bias for double precision format is 1023)

Therefore, B = -1.1_2*2^1.

Therefore, A * B = 1.1001_2*2^4 * (–)1.1_2*2^1 = –(1.1001_2 + 0.11001_2)$*2^5$ = -10.01011_2*2^5 = $-1.0010110*2^6$.

Converting the sum to single precision format, we get

Sign bit = 1 as the number is negative.

Exponent = Actual exponent + Bias. Bias for the single precision format = 127.

Exponent = 6 + 127 = 133 = 128 + 4 + 1 = 10000101_2.

Mantissa = 0010110_2.

Therefore, the number in single precision format is 1 10000101 0010110_2 = 1100 0010 1001 0110_2 = 0xC2960000.

5)The number A = 2^{24} + 1 cannot be represented accurately in a 32-bit single precision floating point representation format but it can be represented in the 32-bit signed integer format accurately. This is because in the 32-bit single precision floating point representation format, the mantissa has only 23 bits and hence A = 1.00…(23 0s)$1*2^{24}$ will be represented as 0x4B800000. (Calculations below)

sign bit = 0, as A is positive

exponent = 24 + 127 = 151 = 128 + 16 + 4 + 2 + 1 = 10010111_2

mantissa = 0.0…(23 0s)1

Therefore, A = 0 10010111 0..(23 0s)$_2$ = 0100 1011 1000..(23 0s)$_2$ = 0x4B800000.

Notice that the last 1 has been lost in this representation as it is the 24th bit which cannot be stored as the mantissa can only store 23 bits. Fairly enough, while converting 0x4B800000 back to an integer, we get

0x4B800000 = 0100 1011 1000…(23 0s) = 0 10010111 0..(23 0s)

Sign bit = 0, hence the number is positive.

Exponent = 10010111_2 = 128 + 16 + 4 + 2 + 1 = 151

Actual exponent = 151 – 127 = 24_{10} as bias = 127 for single float representation.

Mantissa = 0..(23 0s).

Therefore, we get 0x4B800000 = 2^{24} != 2^{24} + 1 = A.

6)Take a = 2^{23}, b = 2^{23} + 1, c = – 1

(a + b) + c = (2^{23} + 2^{23} + 1) – 1 = (2^{24} + 1) – 1 = 2^{24} – 1 (We saw in the previous exam that 2^{24} + 1 is stored as 2^{24} only due to loss of info of the last bit).

a + (b + c) = 2^{23} + (2^{23} + 1 – 1) = 2^{23} + 2^{23} = 2^{24}.

Therefore, addition is not associative for floating point numbers.